

- COURSE: Master calculus 1 using Python: derivatives and applications

SECTION: Applications

LECTURE: CodeChallenge: Gradient descent

TEACHER: Mike X Cohen, sincxpress.com

COURSE URL: udemy.com/course/pycalc1_x/?couponCode=202307

Collected and Edified by Prof Happy AI, 2026

Start coding or generate with AI.

```
# import all necessary modules
import numpy as np
import sympy as sym
import matplotlib.pyplot as plt

import matplotlib_inline.backend_inline
matplotlib_inline.backend_inline.set_matplotlib_formats('svg')
```

Start coding or generate with AI.

- Exercise 1: The function and its derivative

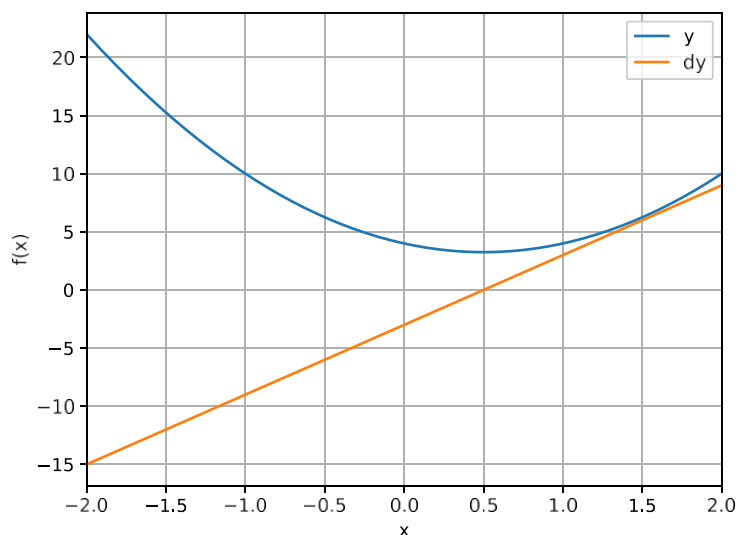
```
# function (as a function)
def fx(x):
    return 3*x**2 - 3*x + 4

# derivative function
def deriv(x):
    return 6*x - 3
```

```
# plot the function and its derivative

# define a range for x
xx = np.linspace(-2,2,2001)

# plotting
plt.plot(xx,fx(xx), xx,deriv(xx))
plt.xlim(xx[[0,-1]])
plt.grid()
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend(['y', 'dy'])
plt.show()
```



Đoạn mã này có chức năng trực quan hóa đồ thị của một hàm số và đạo hàm của nó trên cùng một hệ tọa độ trong một khoảng xác định. Cụ thể, các bước thực hiện bao gồm: Xác định miền giá trị cho biến x : Sử dụng `np.linspace(-2, 2, 2001)` để tạo ra một mảng gồm 2001 điểm dữ liệu cách đều nhau trong khoảng từ -2 đến 2 . Việc sử dụng số lượng điểm lớn giúp các đường cong trên đồ thị trông mượt mà hơn. Vẽ đồ thị chính: Lệnh `plt.plot(xx, fx(xx), xx, deriv(xx))` thực hiện vẽ đồng thời hai đường: hàm số $f(x)$ và đạo hàm của nó $f'(x)$ (được đại diện bởi hàm `deriv`). Cấu hình hiển thị: Giới hạn trục X: `plt.xlim(xx[[0,-1]])` đảm bảo khung hình đồ thị vừa khít với dải giá trị từ -2 đến 2 đã tạo. Lưới tọa độ: `plt.grid()` thêm các đường kẻ ô để người xem dễ dàng đối chiếu và đọc giá trị trên đồ thị. Nhãn trục: `plt.xlabel` và `plt.ylabel` dùng để đặt tên cho trục hoành (x) và trục tung ($f(x)$). Chú thích (Legend): `plt.legend(['y','dy'])` giúp phân biệt rõ đâu là đường biểu diễn hàm số (y) và đâu là đường biểu diễn đạo hàm (dy). Hiển thị kết quả: Lệnh `plt.show()` xuất biểu đồ hoàn chỉnh ra màn hình.

Start coding or generate with AI.

Exercise 2: Gradient descent

```
# random starting point
localmin = np.random.choice(xx,1)
print(f'Started at x = {localmin[0]:.3f}')

# learning parameters
learning_rate = .01
training_epochs = 100

# run through training
for i in range(training_epochs):
    grad = deriv(localmin)
    localmin = localmin - learning_rate*grad

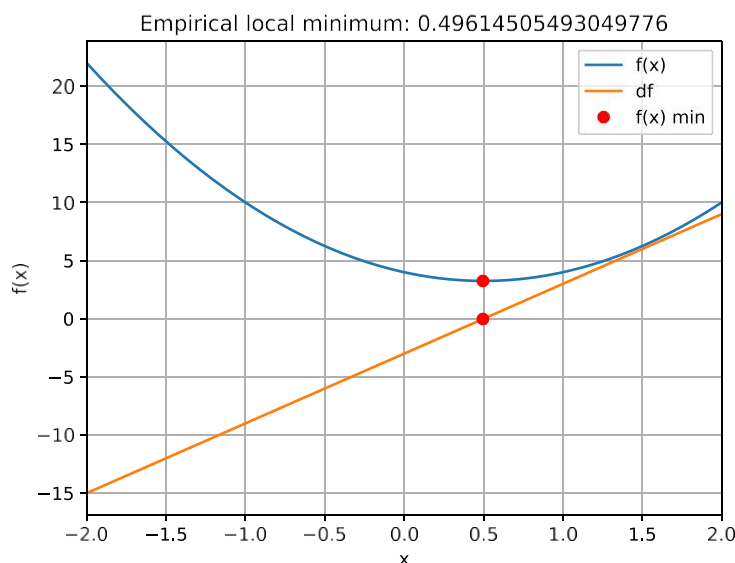
print(f'Finished at x = {localmin[0]:.3f}')
```

Started at x = -1.376
Finished at x = 0.496

```
# plot the results

plt.plot(xx,fx(xx), xx,deriv(xx))
plt.plot(localmin,deriv(localmin),'ro')
plt.plot(localmin,fx(localmin),'ro')

plt.xlim(xx[[0,-1]])
plt.grid()
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend(['f(x)', 'df', 'f(x) min'])
plt.title('Empirical local minimum: %s'%localmin[0])
plt.show()
```



Đoạn mã này có chức năng trực quan hóa vị trí của điểm cực tiểu địa phương tìm được trên đồ thị của cả hàm số và đạo hàm. Cụ thể, các thành phần trong code thực hiện các nhiệm vụ sau: Vẽ đường biểu diễn: Câu lệnh `plt.plot(xx, fx(xx), xx, deriv(xx))` vẽ hai đường cong cơ bản đại diện cho hàm số $f(x)$ và đạo hàm của nó trên miền giá trị xx . Đánh dấu điểm cực tiểu: Hai lệnh `plt.plot(localmin, ..., 'ro')` được

sử dụng để vẽ các điểm tròn màu đỏ ('r' là red, 'o' là hình tròn) tại tọa độ cực tiểu tìm được. Một điểm được chấm trên đường đạo hàm và một điểm được chấm trên đường hàm số để xác định vị trí tương ứng của localmin. Định dạng trục và lưới: `plt.xlim(xx[[0,-1]])`: Giới hạn trục hoành theo phạm vi của dữ liệu xx. `plt.grid()`: Hiển thị lưới để dễ dàng quan sát tọa độ. `plt.xlabel('x')` và `plt.ylabel('f(x)')`: Gán nhãn cho các trục tọa độ. Chú thích và tiêu đề: `plt.legend(['f(x)', 'df', 'f(x) min'])`: Thêm bảng chú thích để người xem phân biệt được đâu là đường hàm số, đường đạo hàm và điểm cực tiểu. `plt.title(...)`: Hiển thị tiêu đề biểu đồ kèm theo giá trị cụ thể của điểm cực tiểu địa phương thu được từ thực nghiệm. Hiển thị: `plt.show()` lệnh cuối cùng để xuất đồ thị ra màn hình.

Start coding or generate with AI.

Exercise 3: Plot the descent

```
# Note: I don't discuss this in the video,
# but it's insightful to modify the parameters
# and observe the effects on the results. For example,
# fix the localmin and adjust the learning rate and epochs!
```

```
# fix the starting point
localmin = -1.5
```

```
# learning parameters
learning_rate = .05
training_epochs = 40
```

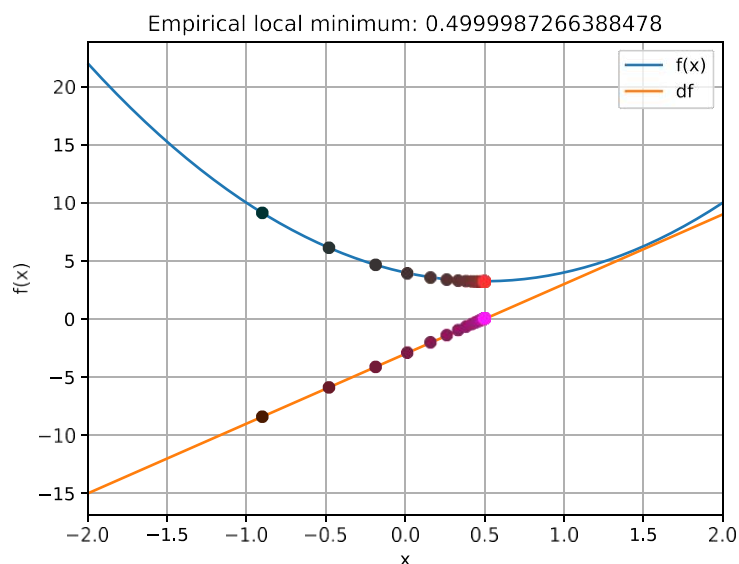
```
# run through training and store all the results
modelparams = np.zeros((training_epochs,2))
for i in range(training_epochs):
    grad = deriv(localmin)
    localmin = localmin - learning_rate*grad
    modelparams[i,0] = localmin
    modelparams[i,1] = grad
```

```
# plot the results
```

```
# plot the function
plt.plot(xx,fx(xx), xx,deriv(xx))
```

```
# plot the progression of points
for i,(xi,dx) in enumerate(modelparams):
    plt.plot(xi,fx(xi), 'o', color=[(i/training_epochs)**(1/2), .2, .2])
    plt.plot(xi,deriv(xi), 'o', color=[(i/training_epochs)**(1/2)*.7+.3, .1, (i/training_epochs)**(1/2)])
```

```
# adjust some plotting aspects
plt.xlim(xx[[0,-1]])
plt.grid()
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend(['f(x)', 'df'])
plt.title('Empirical local minimum: %s'%modelparams[-1,0])
plt.show()
```



Đoạn mã này có chức năng trực quan hóa kết quả của thuật toán tối ưu hóa (thường là Gradient Descent) bằng cách vẽ lại toàn bộ quá trình tìm kiếm điểm cực tiểu địa phương trên đồ thị. Dưới đây là các chức năng cụ thể của từng phần: Vẽ hàm số và đạo hàm: Sử dụng `plt.plot(xx, fx(xx), xx, deriv(xx))` để vẽ hai đường cong cơ bản: hàm mục tiêu $f(x)$ và hàm đạo hàm $f'(x)$ trên cùng một hệ tọa độ. Vẽ quá trình di chuyển của các điểm (Progression): Sử dụng vòng lặp `for` để duyệt qua danh sách `modelparams` (nơi lưu trữ vị trí x và giá trị đạo hàm tại mỗi bước lặp). Tại mỗi bước, một điểm tròn 'o' được vẽ lên đồ thị của hàm số và đạo hàm. Hiệu ứng màu sắc: Màu sắc của các điểm được tính toán dựa trên chỉ số vòng lặp i , giúp tạo hiệu ứng chuyển màu từ nhạt sang đậm (hoặc thay đổi sắc thái) theo thời gian. Điều này giúp người xem nhận biết được thứ tự các bước đi của thuật toán. Cấu hình và định dạng đồ thị: `plt.xlim(xx[[0,-1]])`: Giới hạn trục x đúng bằng phạm vi dữ liệu đã xác định. `plt.grid()`: Thêm lưới tọa độ để dễ quan sát giá trị. `plt.legend(['f(x)', 'df'])`: Thêm chú thích để phân biệt đường biểu diễn hàm số và đường đạo hàm. Hiển thị kết quả cuối cùng: `plt.title(...)`: Hiển thị tiêu đề đồ thị kèm theo giá trị x của điểm cuối cùng trong danh sách `modelparams` (`modelparams[-1,0]`), đây chính là giá trị cực tiểu địa phương mà thuật toán tìm được. `plt.show()`: Xuất đồ thị hoàn chỉnh ra màn hình.

Start coding or generate with AI.

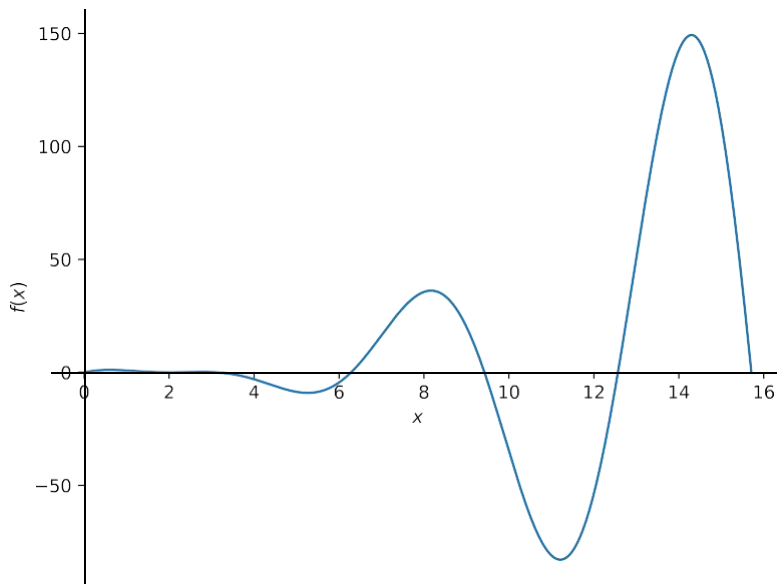
Start coding or generate with AI.

◦ Exercise 4: Now with sympy

```
x = sym.var('x')

# define the function and its derivative
fx = sym.sin(x) * (x-2)**2
df = sym.diff(fx,x)

# quick plot
sym.plot(fx, (x,0,5*sym.pi));
```



Đoạn mã này thực hiện các chức năng chính liên quan đến tính toán ký hiệu (symbolic computation) và trực quan hóa hàm số bằng thư viện SymPy: Khai báo biến ký hiệu: Lệnh `x = sym.var('x')` dùng để định nghĩa x là một biến toán học, cho phép thực hiện các phép toán đại số thay vì tính toán số đơn thuần. Định nghĩa hàm số: Thiết lập biểu thức toán học $f(x) = \sin(x) \cdot (x - 2)^2$. Đây là một hàm số kết hợp giữa hàm lượng giác và một hàm bậc hai. Tính đạo hàm tự động: Lệnh `df = sym.diff(fx, x)` tự động tính toán đạo hàm bậc nhất của hàm số $f(x)$ theo biến x . Vẽ đồ thị nhanh: Sử dụng hàm `sym.plot` để tạo biểu đồ của $f(x)$ trong khoảng giá trị của x từ 0 đến 5π . Đây là công cụ vẽ tích hợp sẵn của SymPy, giúp xem nhanh hình dạng hàm số mà không cần thiết lập phức tạp qua Matplotlib. Tóm lại: Đoạn mã này là bước chuẩn bị dữ liệu toán học (hàm số và đạo hàm) và kiểm tra hình dạng của hàm số trước khi thực hiện các thuật toán tối ưu hóa như Gradient Descent.

```
# lambdify the functions (easier for plotting)
fx_lam = sym.lambdify(x,fx)
df_lam = sym.lambdify(x,df)

# a nicer-looking plot
xx = np.linspace(0,5*np.pi,2001)

_,axs = plt.subplots(2,1,figsize=(8,6))
axs[0].plot(xx,fx_lam(xx),linewidth=3)
```

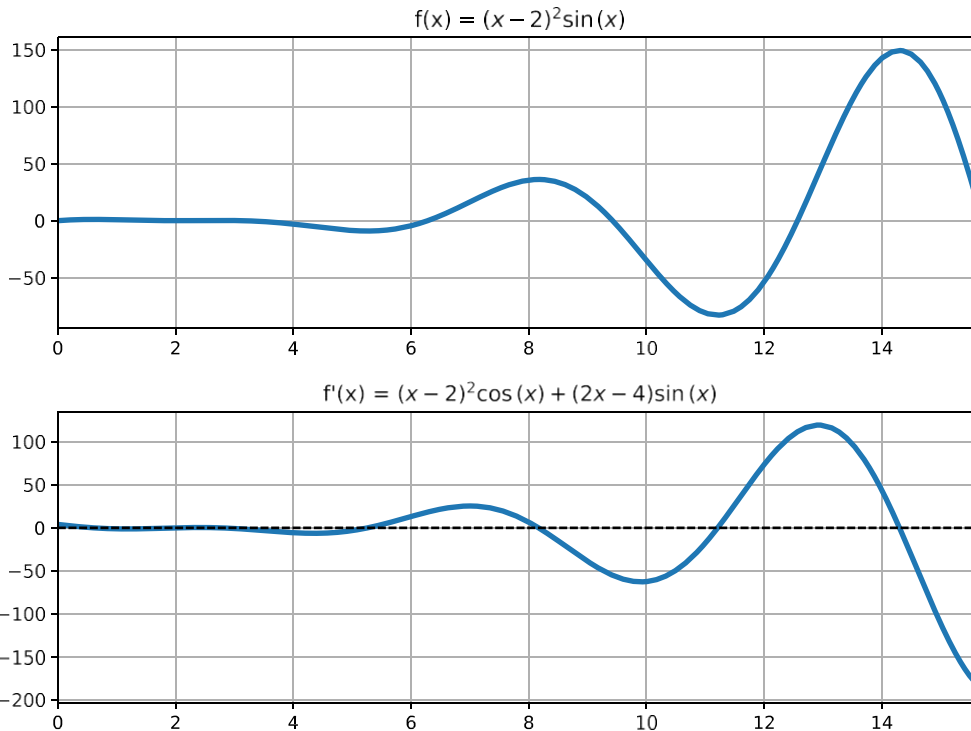
```

axs[0].set_xlim(xx[[0,-1]])
axs[0].grid()
axs[0].set_title(f'f(x) = ${sym.latex(fx)}$')

axs[1].plot(xx,df_lam(xx),linewidth=3)
axs[1].plot(xx[[0,-1]],[0,0],'k--')
axs[1].set_xlim(xx[[0,-1]])
axs[1].grid()
axs[1].set_title(f"f'(x) = ${sym.latex(df)}$")

plt.tight_layout()
plt.show()

```



Đoạn mã này có chức năng chính là biến đổi các biểu thức toán học ký hiệu thành hàm số thực thi được và vẽ đồ thị so sánh giữa hàm số gốc $f(x)$ và đạo hàm của nó $f'(x)$. Dưới đây là chi tiết các chức năng chính dựa trên mã nguồn: Chuyển đổi ký hiệu sang số thực: Sử dụng hàm `sym.lambdify` để biến biểu thức SymPy (`fx` và `df`) thành các hàm Python (`fx_lam`, `df_lam`) có khả năng tính toán nhanh trên các mảng dữ liệu NumPy. Khởi tạo dữ liệu mẫu: Tạo một mảng `xx` chứa 2001 điểm dữ liệu trong khoảng từ 0 đến 5π để đảm bảo đường cong đồ thị được hiển thị mượt mà. Thiết kế bố cục đồ thị đôi: Tạo ra hai biểu đồ con (subplots) nằm chồng lên nhau để người xem dễ dàng quan sát mối tương quan giữa hàm số và đạo hàm của nó. Hiển thị công thức toán học chuyên nghiệp: Sử dụng `sym.latex` để chuyển đổi các biểu thức toán học sang định dạng LaTeX, giúp tiêu đề của đồ thị hiển thị các ký tự toán học (như số mũ, căn thức) một cách đẹp mắt. Vẽ đường tham chiếu đạo hàm: Trong đồ thị đạo hàm ($f'(x)$), mã nguồn vẽ thêm một đường thẳng đứt nét màu đen tại vị trí $y = 0$ ('k--') để xác định các điểm cực trị của hàm số gốc (nơi đạo hàm bằng 0). Tối ưu hóa hiển thị: Sử dụng `plt.tight_layout()` để tự động căn chỉnh khoảng cách, tránh việc các tiêu đề hoặc nhãn trục bị đè lên nhau.

Start coding or generate with AI.

◦ Exercise 5: Gradient descent in sympy

```

# random starting point in x domain
localmin = np.random.choice(xx,1)[0]

# learning parameters
learning_rate = .05
training_epochs = 40

# setup the plot with functions
plt.figure(figsize=(8,6))
plt.plot(xx,fx_lam(xx),label='f(x)')
plt.plot(xx,df_lam(xx),label="f'(x)")

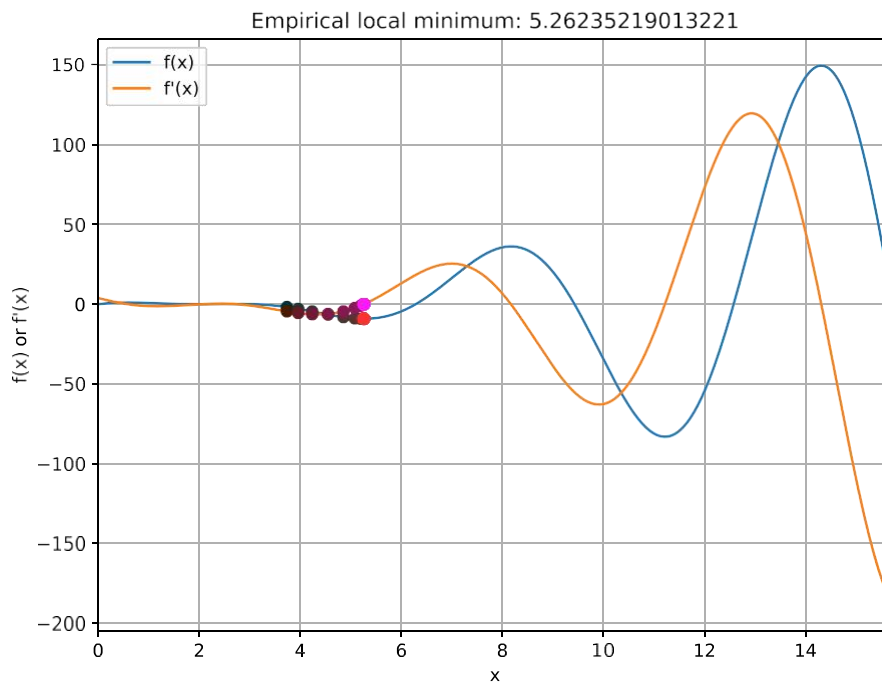
# run through training and plot immediately
for i in range(training_epochs):

```

```
# implement gradient descent
grad = df.subs(x,localmin)
localmin = localmin - learning_rate*grad

# plot the current result
plt.plot(localmin,fx.subs(x,localmin),'o',color=[(i/training_epochs)**(1/2),.2,.2])
plt.plot(localmin,df.subs(x,localmin),'o',color=[(i/training_epochs)**(1/2)*.7+.3,.1,(i/training_epochs)**(1/2)])

plt.grid()
plt.xlabel('x')
plt.ylabel("f(x) or f'(x)")
plt.xlim(xx[[0,-1]])
plt.legend()
plt.title('Empirical local minimum: %s'%localmin)
plt.show()
```



Đoạn mã này thực hiện chức năng mô phỏng thuật toán Gradient Descent (Cực tiểu hóa hàm số) để tìm giá trị cực tiểu địa phương (local minimum) của một hàm số cho trước. Dưới đây là các chức năng chi tiết của đoạn mã: Khởi tạo điểm bắt đầu ngẫu nhiên: Sử dụng `np.random.choice` để chọn ngẫu nhiên một giá trị trong miền `xx` làm điểm xuất phát cho quá trình tìm cực tiểu. Thiết lập tham số học tập: Định nghĩa tốc độ học (`learning_rate = .05`) để kiểm soát độ lớn của mỗi bước di chuyển và số vòng lặp huấn luyện (`training_epochs = 40`). Vẽ khung đồ thị cơ bản: Hiển thị đường cong của hàm số $f(x)$ và đạo hàm $f'(x)$ để người dùng có thể quan sát trực quan sự thay đổi. Thực thi thuật toán Gradient Descent: Trong mỗi vòng lặp, mã sẽ tính toán độ dốc (đạo hàm) tại điểm hiện tại (`df.subs(x, localmin)`) và cập nhật vị trí mới theo công thức: $localmin = localmin - learning_rate \times grad$. Minh họa quá trình học bằng màu sắc: Mỗi bước di chuyển được vẽ trực tiếp lên đồ thị dưới dạng các điểm tròn 'o'. Màu sắc của các điểm thay đổi theo thời gian (sử dụng công thức liên quan đến `i/training_epochs`), giúp người xem thấy được trình tự di chuyển từ điểm bắt đầu đến điểm kết thúc. Hiển thị kết quả cuối cùng: Sau 40 vòng lặp, tiêu đề đồ thị sẽ hiển thị giá trị x tại vị trí mà thuật toán cho là cực tiểu địa phương (Empirical local minimum). Tóm lại: Đây là một công cụ trực quan hóa giúp người học hiểu cách thuật toán Gradient Descent "dò dẫm" tìm điểm thấp nhất trên một bề mặt hàm số dựa vào độ dốc của đạo hàm.

Start coding or generate with AI.

